

1. What is RDD?

RDD stands for **Resilient Distributed Dataset**. It is the most fundamental data structure in Apache Spark. An RDD is essentially a read-only collection of objects that is distributed across the nodes of a cluster, allowing Spark to process data in parallel across many machines at once.

The best way to think of an RDD is as a **list that lives across many computers** at the same time. Instead of all the data sitting on one machine, Spark breaks it into smaller chunks called **partitions**, and each partition lives on a different worker node in the cluster.

Breaking down each word

Word	What it actually means
Resilient	If a node crashes and a partition is lost, Spark can rebuild it automatically. It remembers how the data was created (called lineage) and replays those steps.
Distributed	The data is split into partitions spread across multiple machines. Each machine works on its own slice simultaneously — this is what gives Spark its speed.
Dataset	Simply a collection of records. These can be numbers, strings, tuples, JSON objects, custom Python objects — anything you want.

How RDD partitioning works

When you load data into an RDD, Spark automatically divides it into partitions. Think of a CSV file with 10 million rows being split into 200 partitions of 50,000 rows each, with each partition assigned to a different executor. All 200 chunks are processed at the same time rather than one after the other.

How to create an RDD

There are three main ways to create an RDD in PySpark:

1. From an existing Python collection

```
from pyspark import SparkContext
sc = SparkContext('local[*]', 'RDD Demo')

# parallelize() distributes the list across partitions
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
rdd = sc.parallelize(numbers, numSlices=4)

# numSlices=4 means 4 partitions
```

```
# Partition 0: [1, 2, 3]
# Partition 1: [4, 5]
# Partition 2: [6, 7, 8]
# Partition 3: [9, 10]

print(rdd.getNumPartitions()) # 4
```

2. From an external file (HDFS, S3, local disk)

```
# Each HDFS block becomes one partition automatically
log_rdd = sc.textFile('hdfs:///data/logs/app.log')

# Can also read multiple files at once
all_logs = sc.textFile('hdfs:///data/logs/*.log')

# Read a whole directory
rdd = sc.wholeTextFiles('hdfs:///data/configs/')
# This gives (filename, file_content) pairs
```

3. By transforming an existing RDD

```
base_rdd = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# filter() creates a new RDD — original is unchanged
even_rdd = base_rdd.filter(lambda x: x % 2 == 0)

# map() transforms each element
squared_rdd = even_rdd.map(lambda x: x ** 2)

# squared_rdd is now: [4, 16, 36, 64, 100]
```

Key properties of an RDD

- Immutable — once created, you cannot change an RDD. You can only create new RDDs from it by applying transformations
- Lazy — nothing is actually computed until you call an action (explained in detail in Section 3)
- Typed — each RDD has a type, like RDD[String] or RDD[Int] in Scala, though Python hides this
- Partitioned — data is split into partitions; you can control how many partitions an RDD has
- Lineage-aware — Spark tracks how each RDD was derived from its parent, enabling fault recovery

2. Advantages of using RDD

RDDs are the foundation of Spark for good reasons. Here are the most important advantages — and more importantly, why each one matters in practice.

Fault Tolerance through Lineage

This is arguably the most important advantage. Traditional distributed systems achieve fault tolerance by **replicating data** — storing 3 copies of everything on different machines. That is expensive in terms of storage and network bandwidth. RDDs take a completely different approach.

Instead of copying data, Spark stores the **recipe for how the RDD was created** — called the **lineage graph** or **RDD lineage**. If a partition is lost because a machine crashes, Spark simply re-executes the relevant steps on the remaining data to recreate that partition.

How lineage recovery actually works

```
# Suppose we build this RDD pipeline:
raw      = sc.textFile('hdfs:///data/sales.csv')    # Step 1
parsed   = raw.map(parse_row)                       # Step 2
filtered = parsed.filter(lambda r: r['amount']>0)   # Step 3
result   = filtered.map(lambda r: (r['region'], r['amount']))

# Spark stores the lineage:
# result <- filtered <- parsed <- raw (file on HDFS)

# If Executor 2 crashes while processing partition 5 of 'filtered',
# Spark re-reads partition 5 of the original file from HDFS,
# re-applies parse_row, re-applies the filter — on a healthy executor.
# No data was lost. No intervention needed.
```

In-Memory Processing

Hadoop MapReduce reads from disk and writes to disk at **every step** of a multi-step job. If you have a 10-step pipeline, your data goes to disk and back 10 times. RDDs in Spark keep intermediate data in **RAM**, which can be 10x–100x faster than disk for iterative workloads like machine learning.

Operation	Hadoop MapReduce	Spark RDD
Step 1 output	Written to HDFS	Kept in memory
Step 2 input	Read from HDFS	Read from memory
Step 3 output	Written to HDFS	Kept in memory
10 iterations (ML)	20 disk reads/writes	1 disk read + 19 memory ops

Caching an RDD for repeated use

```
training_data = sc.textFile('hdfs:///ml/training.txt') \
    .map(parse_features) \
    .filter(lambda x: x is not None)

# Without cache: every iteration reads from disk and re-parses
# With cache: first iteration computes and stores in RAM,
# all subsequent iterations read from RAM
training_data.cache() # or training_data.persist()

for iteration in range(100):
    gradient = training_data.map(compute_gradient).reduce(sum_gradients)
    update_weights(gradient)
    # After iteration 1, training_data comes from cache — very fast
```

Parallel Processing

Because an RDD is split into partitions and each partition is processed independently, Spark can run operations on all partitions at the same time. The degree of parallelism is limited by the number of partitions and the number of available CPU cores in the cluster.

You can control parallelism explicitly. A good rule of thumb is **2–4 partitions per CPU core** available in your cluster.

```
# Controlling number of partitions
rdd = sc.parallelize(data, numSlices=200) # 200 partitions at creation

# Repartition after the fact — triggers a shuffle
rdd_more = rdd.repartition(400) # more partitions, more parallelism
rdd_less = rdd.coalesce(50) # fewer partitions, no full shuffle

# Check current partition count
print(rdd.getNumPartitions())

# See which elements are in which partition
print(rdd.glom().collect())
```

Immutability makes debugging easier

Because RDDs cannot be modified in place, you always have a clear chain of transformations. If something goes wrong, you can inspect each step independently. There are no hidden side effects — a function applied to an RDD partition cannot accidentally modify the original data.

Flexibility — works with any Python object

Unlike SQL tables or DataFrames which require a fixed schema, an RDD can hold any Python object — custom classes, dictionaries, nested lists, binary data, whatever your problem needs. This makes RDDs the right tool when your data does not fit neatly into rows and columns.

```
# RDD of raw strings
text_rdd = sc.textFile('logs.txt')

# RDD of Python dicts
dict_rdd = sc.parallelize([
    {'name': 'Alice', 'score': 95, 'tags': ['python', 'spark']},
    {'name': 'Bob', 'score': 87, 'tags': ['java']},
])

# RDD of custom objects
class SensorReading:
    def __init__(self, device_id, temp, timestamp): ...

sensor_rdd = sc.parallelize([
    SensorReading('dev1', 22.4, 1700000000),
    SensorReading('dev2', 19.1, 1700000001),
])
```

Quick summary of advantages

Advantage	Why it matters
Fault tolerance via lineage	No need to store extra copies of data; lost partitions are recomputed automatically
In-memory computation	10–100x faster than Hadoop MapReduce for multi-step and iterative jobs
Parallel processing	All partitions processed simultaneously — linear scalability as you add nodes
Immutability	No accidental data corruption; easy to trace bugs through the lineage chain
Schema flexibility	Works with any Python object, not just structured rows and columns
Unified API	Same RDD model works whether running locally or on a 1000-node cluster

3. What is Lazy Evaluation?

Lazy evaluation is one of the most important concepts in Spark, and it is the reason why Spark can be so efficient. The idea is simple: **Spark does not execute any computation the moment you write a transformation. It waits.**

Spark only starts doing actual work when you call an **action** — something that needs a real result, like counting rows, printing values, or saving to disk. Until then, every transformation you write just gets added to a **plan** that Spark is building in the background.

Transformations vs Actions — the key distinction

This is the line between lazy and eager:

Type	Examples	When does it execute?
Transformation (lazy)	map, filter, flatMap, groupBy, join, select, union	Never — just adds to the plan
Action (eager)	count, collect, show, save, first, take(n), reduce	Immediately — triggers the whole plan

A concrete example — step by step

```
sc = SparkContext('local[*]', 'Lazy Demo')

# Step 1: Load file — LAZY (no data is read yet)
rdd = sc.textFile('hdfs:///data/access_logs.txt')
print('After textFile — nothing happened yet')

# Step 2: Filter — LAZY (no work done)
errors = rdd.filter(lambda line: 'ERROR' in line)
print('After filter — still nothing happened')

# Step 3: Extract fields — LAZY
error_codes = errors.map(lambda line: line.split()[2])
print('After map — still nothing happened')

# Step 4: count() is an ACTION — NOW Spark does everything above
total = error_codes.count()
print(f'Total errors: {total}')
# At this point, Spark:
#   1. Reads the file from HDFS
#   2. Filters for lines containing ERROR
#   3. Extracts field index 2 from each
#   4. Counts the results
```

```
# 5. Returns the number to the driver
```

Why does lazy evaluation help?

1. Spark can optimize the whole plan before running it

Because Spark sees all your transformations upfront (before running any), it can apply optimizations that would be impossible if each step ran immediately. The most important one is **predicate pushdown** — moving filters as early as possible to reduce data volume.

```
# You write this:
result = rdd \
    .map(expensive_parse)      # parse every record
    .filter(lambda r: r['country'] == 'IN') # then filter
    .map(compute_stats)        # then process

# Spark's optimizer rewrites the plan to:
# filter FIRST (skip records where country != IN),
# THEN parse (only parse the records we need),
# THEN compute stats
#
# If only 5% of records have country='IN', this means
# expensive_parse runs on 5% of the data instead of 100%.
```

2. Unnecessary work is skipped entirely

If you define 10 transformations but your action only needs the first 3 steps to produce a result, Spark only executes those 3 steps. The other 7 are never computed.

3. Multiple actions on the same data can share computation

```
filtered = rdd.filter(lambda x: x > 0).map(transform)

# Cache the result so it is computed only once
filtered.cache()

# Both actions below reuse the cached result
total    = filtered.count()      # Action 1 — computes and caches
maximum  = filtered.max()        # Action 2 — reads from cache
minimum  = filtered.min()        # Action 3 — reads from cache

# Without cache, the filter+transform would run 3 times.
# With cache, it runs once.
```

The DAG — what lazy evaluation actually builds

As you chain transformations, Spark builds a **DAG (Directed Acyclic Graph)** — a logical description of all the steps. Think of it as a flowchart of your data pipeline. When an action is triggered, the DAG Scheduler looks at this graph, finds the most efficient way to execute it, divides the work into stages, and sends tasks to executors.

```
# Visualising the DAG concept (not runnable code):

#   textFile --> [RDD: raw lines]
#           |
#           filter(ERROR)
#           |
#   [RDD: error lines]
#           |
#           map(extract code)
#           |
#   [RDD: error codes]
#           |
#           count()    <--- ACTION: DAG executes HERE

# You can view the DAG in Spark UI at http://localhost:4040
# Click on a job -> DAG Visualization
```

Common mistake — calling actions too early

One of the most frequent performance mistakes is calling `collect()` or other actions repeatedly inside loops, when you could structure the code to call the action once.

```
# BAD: collect() inside a loop — triggers full computation each time
for threshold in [10, 20, 30, 40, 50]:
    result = rdd.filter(lambda x: x > threshold).collect()
    process(result)  # 5 full Spark jobs run

# BETTER: do the filtering in Python after one collect()
all_data = rdd.collect()  # One Spark job
for threshold in [10, 20, 30, 40, 50]:
    result = [x for x in all_data if x > threshold]
    process(result)  # Pure Python, no Spark overhead

# BEST (for large data): use a single parameterised Spark job
results = rdd.filter(lambda x: x > 10).cache()
# then derive further RDDs from results without re-reading raw data
```

Summary — how the three concepts connect

These three concepts are not independent — they work together as a system:

Concept	How it connects to the others
RDD	The distributed data structure that transformations and actions operate on. Immutability makes lazy evaluation safe — there are no side effects to worry about.
Advantages of RDD	Fault tolerance relies on lineage, which is only possible because of lazy evaluation — Spark has to know the full recipe before running anything. In-memory speed is enabled by lazy evaluation's ability to optimise the full pipeline.
Lazy Evaluation	Makes both of the above possible. Without it, Spark could not build the DAG, could not optimise, and could not recover from faults using lineage — it would just be a regular parallel processing library.
